

RESPONSIVE CPU TASK SCHEDULER

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of
CSA 0428 – Operating System for Real Time Computing

to the award of the degree of
BACHELOR OF ENGINEERING
IN
Computer Science and Engineering

Submitted by

Mohamed Rayhaan Khan (192511006)

Deepak. R (192511008)

Emad Farooqui (192511019)

Vijay. A (192511030)

Janikaa Sri. R (192511092)

Sandhiya R (192511132)

R Tamilarasan (192511298)

Under the Supervision of

Dr. M. Anoop



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

February 2026



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We, **Mohamed Rayhaan Khan (192511006), Deepak. R (192511008), Emad Farooqui (192511019), Vijay. A (192511030), Janikaa Sri. R (192511092), Sandhiya R (192511132), R Tamilarasan (192511298)** of the **B.E-CSE, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai**, hereby declare that the Capstone Project Work entitled “**Responsive CPU Task Scheduler**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Date:

Signature

Mohamed Rayhaan Khan (192511006),
Deepak. R (192511008),
Emad Farooqui (192511019),
Vijay. A (192511030),
Janikaa Sri. R (192511092),
Sandhiya R (192511132),
R Tamilarasan (192511298)



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Responsive CPU Task Scheduler**” has been carried out by **Mohamed Rayhaan Khan (192511006), Deepak. R (192511008), Emad Farooqui (192511019), Vijay. A (192511030), Janikaa Sri. R (192511092), Sandhiya R (192511132), R Tamilarasan (192511298)** under the supervision of **Dr.M.Anoop** and is submitted in partial fulfilment of the requirements for the current semester of the B.E. CSE program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. S. Anushya
Program Director

Computer Science and Engineering
Saveetha School of Engineering
SIMATS

SIGNATURE

Dr. M. Anoop
Associate Professor

Computer Science and Engineering
Saveetha School of Engineering
SIMATS

Submitted for the Project work Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Program Director, Dr.S.Anushya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr.M.Anoop for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

ABSTRACT

CPU scheduling is a fundamental function of an operating system that determines the execution order of processes to achieve efficient utilization of processor resources. Traditional scheduling algorithms such as First-Come-First-Serve (FCFS) and Round Robin (RR) operate using static rules and fixed parameters, which often fail to perform efficiently under dynamic and mixed workloads. These limitations result in increased waiting time, poor turnaround time, and reduced system responsiveness in modern computing environments. This project presents the design and implementation of an AI-based CPU Scheduling Simulator that integrates classical scheduling techniques with machine learning to develop an intelligent and adaptive scheduling mechanism. The proposed system implements a custom scheduler named V-RTS-ML (Variable Responsive Task Scheduler – Machine Learning based), which uses Decision Tree algorithms to classify processes as short or long and dynamically predict optimal time quantum values. The simulator provides an interactive graphical user interface developed using Python Tkinter, real-time process state visualization, Gantt chart generation, and performance comparison graphs. Experimental evaluation of the system demonstrates that the ML-assisted scheduler improves scheduling efficiency by reducing average waiting time and turnaround time compared to traditional algorithms. This project highlights the potential of machine learning in enhancing operating system scheduling techniques and serves as an effective educational tool for understanding intelligent CPU scheduling.

TABLE OF CONTENTS

S.NO	SUB-TOPICS	PAGES
1	Introduction	1-4
	1.1 Background Information	
	1.2 Project Objectives	
	1.3 Significance	
	1.4 Scope	
	1.5 Methodology Overview	
2	Problem Identification and Analysis	5-7
	2.1 Existing System	
	2.2 Limitations of the Existing System	
	2.3 Proposed System	
	2.4 System Requirements	
	2.5 Feasibility Study	
3	Solution Design and Implementation	8-13
	3.1 Development and Design Process	
	3.2 Tools and Technologies Used	
	3.3 Solution Overview	
	3.4 Engineering Standards Applied	
	3.5 Solution Justification	
	3.6 Modules Description	
4	Results and Recommendations	14-16
	4.1 Overview of Implementation Results	
	4.2 Results of FCFS Scheduling	
	4.3 Results of Round Robin Scheduling	
	4.4 Results of V-RTS-ML Scheduling	
5	Reflection on Learning and Personal Development	17-19
	5.1 Key Learning Outcomes	
	5.2 Challenges Encountered and Overcome	
	5.3 Application of Engineering Standards	
	5.4 Insights into the Industry	
6	Conclusion	20
7	Future Enhancements	21
8	References	22
9	Appendices	23-25

TABLE OF FIGURES

S.NO	FIGURE	PAGE NO
1	Figure 3.1: Smart Segmentation	11
2	Figure 3.2: RTS	13
3	Figure 3.3: FCFS	14
4	Figure 4.1: Performance Analysis	16

Chapter 1: Introduction

1.1 Background Information

In modern computer systems, the Central Processing Unit (CPU) is the most critical and highly utilized hardware resource. Every application, program, and system process requires CPU time for execution. Since multiple processes compete for the processor simultaneously, the operating system must decide the order in which these processes are executed. This decision-making mechanism is known as CPU scheduling. Efficient CPU scheduling is essential for maximizing CPU utilization, reducing waiting time, minimizing turnaround time, and ensuring fair allocation of processor resources among processes.

Over the years, several traditional CPU scheduling algorithms have been developed to manage process execution. Algorithms such as First-Come-First-Serve (FCFS) and Round Robin (RR) are commonly used in operating systems to control the execution of processes. These algorithms follow fixed rules and predefined parameters to make scheduling decisions. Although these methods are simple and easy to implement, they were designed for earlier computing environments where workloads were relatively predictable and less complex.

In present-day computing systems, workloads are highly dynamic and unpredictable. Processes vary widely in terms of arrival time, burst time, and priority. Static scheduling techniques are not able to adapt to such variations in real time. As a result, traditional algorithms often lead to increased waiting time, inefficient CPU utilization, and poor system responsiveness. To overcome these limitations, intelligent and adaptive scheduling mechanisms are required.

With recent advancements in Artificial Intelligence and Machine Learning, it has become possible to enhance traditional operating system functions using data-driven decision making. Machine learning techniques can analyze process behavior and dynamically predict optimal scheduling strategies. The proposed system implements an AI-based CPU Scheduling Simulator that demonstrates how machine learning can improve scheduling efficiency in modern computing environments.

1.2 Project Objectives

The primary objective of this project is to design and implement an intelligent CPU scheduling simulator that combines classical scheduling algorithms with machine learning techniques. The project aims to develop a system capable of analyzing process attributes such as arrival time and burst time and making adaptive scheduling decisions dynamically. Another important objective is to provide an interactive and user-friendly platform for visualizing and comparing different CPU scheduling algorithms.

The project specifically focuses on implementing traditional scheduling methods such as FCFS and Round Robin and enhancing them with a custom machine learning–based scheduler named RTS-ML (Responsive Task Segmentation Scheduler – Machine Learning based). The objective of this scheduler is to classify processes intelligently as short or long and to predict an optimal time quantum using Decision Tree algorithms. In addition, the project aims to generate Gantt charts, performance graphs, and comparison tables to evaluate the effectiveness of different scheduling techniques. Overall, the objective is to demonstrate how artificial intelligence can be practically applied to improve core operating system scheduling mechanisms.

1.3 Significance

The significance of this project lies in its ability to introduce intelligence and adaptability into CPU scheduling. Traditional scheduling algorithms are static in nature and do not adjust their behavior based on changing workloads. This often results in inefficient scheduling decisions, especially in systems with mixed types of processes. The proposed ML-assisted scheduler addresses this limitation by analyzing process characteristics and making smarter scheduling decisions in real time.

This project is highly valuable from an educational perspective as well. Understanding CPU scheduling is a fundamental part of operating system studies, but most learning methods remain theoretical. The developed simulator provides an interactive platform where users can visualize scheduling behavior, observe Gantt charts, and compare algorithm performance. This makes complex operating system concepts easier to understand.

Furthermore, the project demonstrates the practical application of machine learning in system-level software. By integrating Decision Tree classification and regression models with scheduling algorithms, the system shows how AI techniques can be used to improve traditional computing methods. Therefore, the project is significant both as an academic learning tool and as a demonstration of intelligent system design.

1.4 Scope

The scope of this project is limited to the simulation and analysis of CPU scheduling techniques in a controlled software environment. The system is designed as a Python-based simulator and does not modify or interact with an actual operating system kernel. The project focuses on implementing and comparing three scheduling approaches: FCFS, Round Robin, and the proposed V-RTS-ML scheduler.

The simulator provides graphical visualization of scheduling behavior using Gantt charts and performance comparison graphs. It also includes an interactive user interface developed with Tkinter, allowing users to input process details and observe results in real time. Machine learning models are trained using predefined datasets to classify processes and predict optimal time quantum values.

The project is limited to single-processor scheduling scenarios and considers only basic process parameters such as arrival time and burst time. Advanced features such as priority scheduling, multi-core scheduling, real-time operating system integration, and kernel-level implementation are beyond the scope of this project. However, the simulator serves as a strong foundation for understanding intelligent CPU scheduling and can be extended further in future work.

1.5 Methodology Overview

The methodology adopted for this project follows a structured and modular approach. The first stage involved studying traditional CPU scheduling algorithms and identifying their limitations in handling dynamic workloads. Based on this analysis, a hybrid scheduling strategy integrating machine learning with classical algorithms was designed.

The system was implemented using the Python programming language. Classical

scheduling algorithms such as FCFS and Round Robin were implemented first to establish baseline performance. Machine learning models were then developed using Decision Tree Classifier and Decision Tree Regressor to classify processes and predict optimal time quantum values. These models were integrated into a custom scheduler named V-RTS-ML.

A graphical user interface was developed using Tkinter to allow users to input process data and visualize scheduling behavior. Matplotlib was used to generate Gantt charts and performance comparison graphs. The system was tested with various input scenarios, and performance metrics such as average waiting time and turnaround time were calculated to evaluate effectiveness.

Finally, the results of all scheduling algorithms were compared to demonstrate the advantages of the proposed ML-based scheduler. This structured methodology ensured systematic development, accurate implementation, and effective evaluation of the intelligent CPU scheduling simulator.

Chapter 2: Problem Identification and Analysis

2.1 Existing System

In traditional operating systems, CPU scheduling is handled using fixed and predefined algorithms. Commonly used scheduling techniques such as First-Come-First-Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR) are implemented in most operating systems to manage process execution. These algorithms follow simple logical rules to determine which process should be executed next. They were designed to provide fairness and efficiency in earlier computing environments where workloads were relatively stable and predictable.

Among these algorithms, FCFS is the simplest scheduling method in which processes are executed strictly in the order of their arrival. Although it is easy to implement, FCFS often results in poor performance because long processes can block short processes, leading to high waiting time and turnaround time. The Round Robin algorithm improves fairness by allocating a fixed time quantum to each process in a cyclic order. However, the efficiency of Round Robin depends heavily on the selection of an appropriate time quantum value. If the time quantum is too large, the algorithm behaves like FCFS, and if it is too small, excessive context switching occurs, reducing system performance.

These traditional scheduling approaches do not consider the dynamic behavior of processes. They treat all processes in a similar manner without analyzing their actual CPU requirements. In real-world computing environments, process workloads vary significantly, and static scheduling strategies are unable to adapt to such variations. As a result, the existing system suffers from inefficiency, lack of flexibility, and poor responsiveness when handling mixed workloads.

2.2 Limitations of the Existing System

Although traditional CPU scheduling algorithms have been widely used for many years, they have several practical limitations. One major drawback is their static nature. These algorithms make scheduling decisions based only on predefined rules and do not adjust

according to changing process characteristics. They do not analyze whether a process is short or long, CPU-intensive or interactive, and therefore cannot optimize scheduling behavior dynamically.

Another limitation is the dependency on fixed parameters. In the case of Round Robin scheduling, the performance of the algorithm is highly dependent on the selected time quantum. Determining an optimal time quantum manually is very difficult, and an improper value can lead to either poor response time or unnecessary context switching overhead. Traditional algorithms also fail to handle real-time variations in workloads, which results in inefficient CPU utilization.

Furthermore, classical scheduling methods lack any form of intelligence or learning capability. They do not use historical process data to improve future scheduling decisions. Because of this, they are not suitable for modern systems that require adaptive and intelligent resource management. These limitations clearly indicate the need for an improved scheduling mechanism that can analyze process behavior and make smarter decisions automatically.

2.3 Proposed System

To overcome the limitations of traditional CPU scheduling techniques, this project proposes an AI-assisted CPU Scheduling Simulator that integrates machine learning with classical scheduling algorithms. The proposed system introduces an intelligent scheduler named V-RTS-ML (Variable Responsive Task Scheduler – Machine Learning based), which is capable of analyzing process characteristics and adapting scheduling behavior dynamically.

In the proposed system, classical scheduling algorithms such as FCFS and Round Robin are implemented as baseline methods. Along with these, the new ML-based scheduler is developed to enhance decision making. The system uses a Decision Tree Classifier to classify processes as short or long based on their arrival time and burst time. A Decision Tree Regressor is used to predict an optimal time quantum value dynamically instead of using a fixed value.

The proposed system is implemented as an interactive simulator using Python programming. A graphical user interface developed with Tkinter allows users to enter process details and observe scheduling behavior in real time. The simulator generates Gantt charts,

performance comparison graphs, and tabular results to evaluate the efficiency of different algorithms. By combining traditional techniques with machine learning, the proposed system provides a more adaptive, intelligent, and efficient approach to CPU scheduling.

2.4 System Requirements

For the successful implementation and execution of the proposed CPU scheduling simulator, certain hardware and software resources are required. The system is designed to run on a standard personal computer without the need for specialized hardware.

The hardware requirements include a computer system with a minimum of 4 GB RAM, an Intel i3 or higher processor, and sufficient storage space to install Python and related libraries. A display device with standard graphical capability is required to visualize the graphical user interface and performance graphs.

The software requirements for the project include the Windows operating system, Python programming language version 3.8 or higher, and essential Python libraries such as Tkinter for GUI development, Matplotlib for graph generation, Scikit-learn for machine learning models, and PIL (Pillow) for image handling. These tools together provide the necessary environment to develop and execute the intelligent CPU scheduling simulator.

2.5 Feasibility Study

Before developing the system, a feasibility study was conducted to analyze whether the proposed project could be implemented successfully. Python provides strong support for GUI development, data visualization, and machine learning, making it an ideal platform for implementing the proposed system.

From an operational point of view, the system is easy to use and does not require any complex installation or configuration. Users can simply input process details and run simulations without needing advanced technical knowledge. Economically, the project is also feasible since it uses open-source software and does not require any additional financial investment. All required libraries are freely available, and the system can run on ordinary personal computers. Therefore, the proposed project is technically, operationally, and economically feasible.

Chapter 3: Solution Design and Implementation

3.1 Development and Design Process

The development of BINSTRUCT followed a structured design process aimed at ensuring modularity, usability, and accuracy in simulating binary-based architectures. The first step involved analysing the theoretical foundations of number systems, instruction encoding, and addressing modes to identify the functional requirements of each module. Based on these requirements, the system was divided into three interconnected modules: Number System Converter, Instruction Encoder and Format Breakdown, and Addressing Mode and Memory Access Simulator.

The design phase emphasized modularity, where each component was developed independently but integrated into a single platform. The Number System Converter was designed using algorithms for base conversion, ensuring support for binary, decimal, hexadecimal, and octal systems. The Instruction Encoder was designed to parse high-level assembly instructions, generate corresponding binary formats, and break them down into opcode and operand fields. Finally, the Addressing Mode Simulator was implemented with a graphical memory model to illustrate how different addressing techniques retrieve or store data. Iterative testing was applied throughout the process to refine functionality and guarantee that the simulator remained both accurate and user-friendly.

3.2 Tools and Technologies Used

To ensure a smooth development process, appropriate tools and technologies were chosen for building the BINSTRUCT platform. The primary programming environment selected was Python, owing to its simplicity, readability, and extensive library support for both algorithmic computation and graphical simulation. For the Number System Converter, Python's arithmetic and string manipulation functions were leveraged to perform precise base conversions. The Instruction Encoder utilized parsing libraries and custom logic to handle binary instruction representations, while visualization components were integrated for the Addressing Mode Simulator using GUI frameworks such as Tkinter or PyQt.

Together, these tools ensured that BINSTRUCT was developed systematically with robust functionality and well-documented features.

3.3 Solution Overview

The proposed solution, BINSTRUCT, provides an integrated platform for learning and simulating core aspects of binary-based architectures. The system is divided into three modules, each addressing a specific educational challenge. The Number System Converter simplifies the process of converting between different numeral systems, thereby strengthening learners' mathematical foundations. The Instruction Encoder and Format Breakdown module bridges the gap between assembly instructions and their binary counterparts by showing opcode segmentation, instruction length, and operand handling. Lastly, the Addressing Mode and Memory Access Simulator offer an interactive visualization of how instructions operate on memory using different addressing schemes, such as immediate, direct, indirect, and register-based modes.

By combining these three modules, BINSTRUCT creates a holistic learning experience that moves beyond static textbook knowledge. It enables users to experiment, observe outcomes in real-time, and build a deeper understanding of instruction-level processes in binary architectures. The solution is designed to be scalable, extendable, and adaptable to different instruction set architectures, making it a valuable educational and training tool.

3.4 Engineering Standards Applied

The development of BINSTRUCT was guided by established engineering standards to ensure quality, accuracy, and reliability. For algorithm design, IEEE standards for binary arithmetic and floating-point representation were considered to maintain correctness in number system conversions. In software development, modular programming principles and software engineering best practices were followed, including proper documentation, version control, and unit testing. User interface design adhered to human-computer interaction (HCI) standards, ensuring clarity, accessibility, and ease of use for both beginners and advanced learners.

Furthermore, the system's architecture followed the principles of structured software design, which promotes modularity and reusability.

3.5 Solution Justification

The decision to implement BINSTRUCT as an educational tool is justified by the pressing need for interactive platforms that simplify the learning of binary-based architectures. Traditional methods often fail to provide practical engagement with number systems, instruction encoding, and addressing modes, leading to gaps in understanding. BINSTRUCT addresses this issue by integrating all three aspects into one coherent platform, offering both computational accuracy and visual simulation.

The inclusion of the Number System Converter ensures that users can build a strong base in arithmetic conversions, while the Instruction Encoder helps bridge the gap between high-level instructions and low-level binary representations. The Addressing Mode Simulator adds further value by demonstrating how data flows within a processor and memory environment. These modules, when combined, create a comprehensive solution that improves comprehension, reduces errors, and enhances practical learning. Therefore, the solution not only meets academic needs but also provides lasting benefits for students, educators, and researchers working in computer architecture and digital systems.

3.6 Modules Description

1. Machine Learning Algorithm Module

The first module of the project is the Machine Learning Algorithm Module. This module acts as the intelligence layer of the entire system. It is responsible for analyzing process attributes such as arrival time and burst time and making smart scheduling decisions. In this module, a Decision Tree Classifier is implemented to classify processes as short or long. Along with this, a Decision Tree Regressor is used to predict an optimal time quantum value dynamically. These machine learning models are trained using sample datasets and integrated into the scheduling system. The main objective of this module is to remove the limitation of fixed scheduling parameters and introduce adaptive decision making into CPU scheduling.

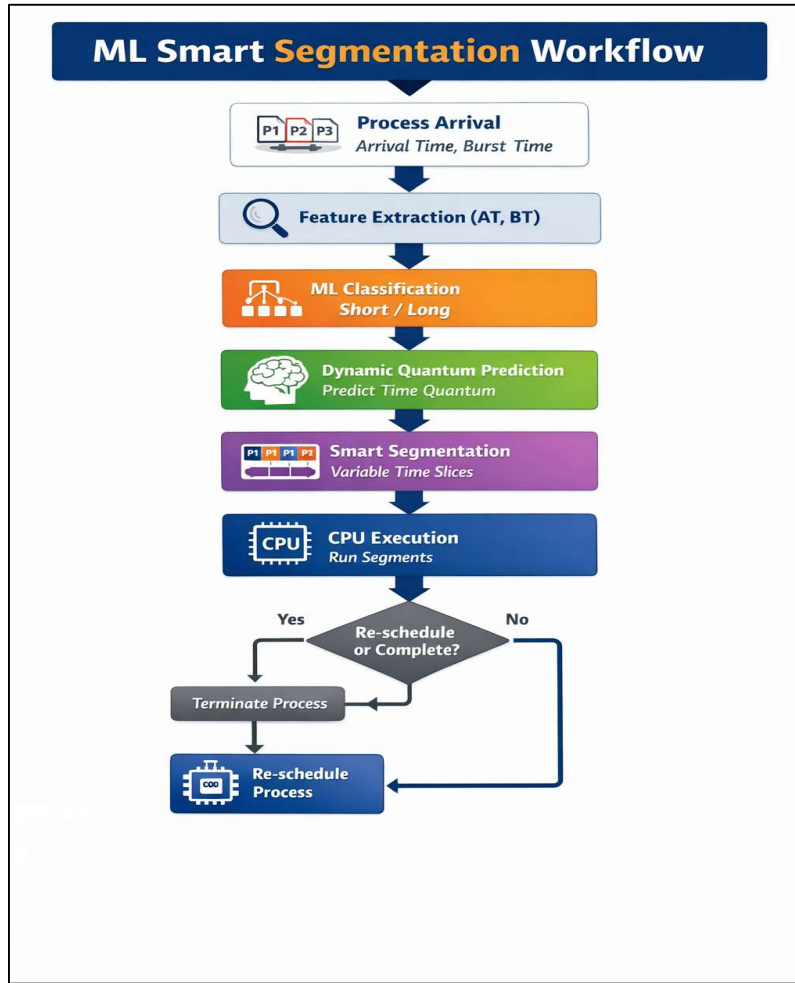


Figure 3.1: Smart Segmentation

2. RTS-ML Scheduler Module

The second module is the RTS-ML Scheduler Module, which implements the proposed V-RTS-ML scheduling algorithm. This module combines traditional scheduling logic with machine learning predictions to perform intelligent scheduling. Based on the classification provided by the ML model, short processes are executed with higher priority, while long processes are scheduled using a Round Robin strategy with dynamically predicted time quantum. This module continuously monitors process execution and updates scheduling decisions accordingly. The RTS-ML scheduler ensures better responsiveness, reduced waiting time, and improved turnaround time compared to classical algorithms.

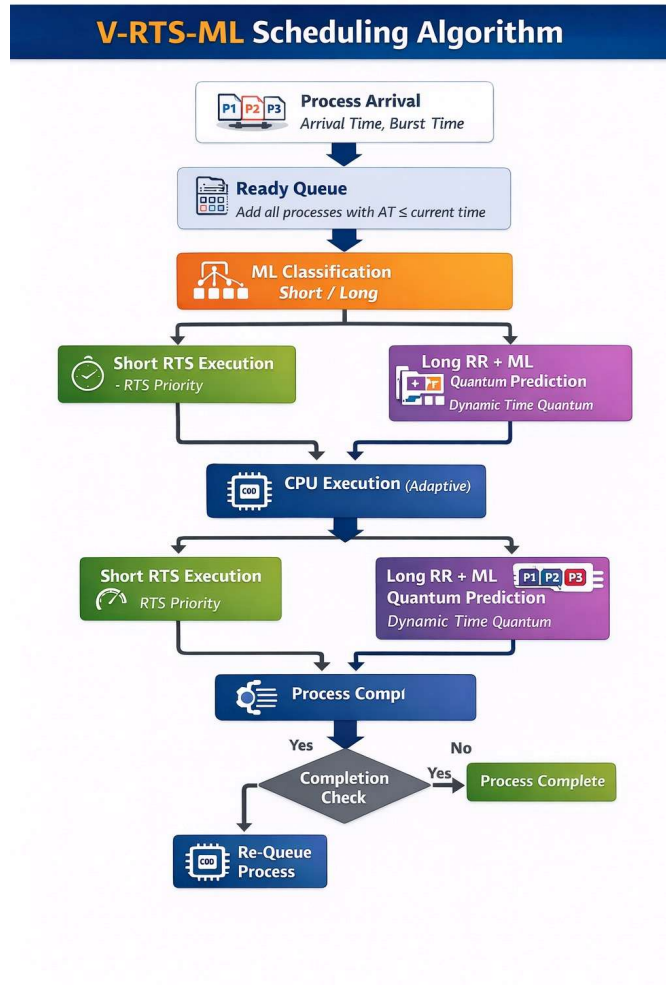


Figure 3.2: RTS

3. Graphical User Interface and Simulation

The third module of the system is the Graphical User Interface and Simulation Module. This module provides an interactive platform through which users can input process details and run scheduling algorithms. The GUI is developed using Python Tkinter and allows users to visualize process execution in real time. It supports Gantt chart generation, live scheduling simulation, and side-by-side comparison of different algorithms. This module makes the project user-friendly and helps users clearly understand the working of FCFS, Round Robin, and V-RTS-ML schedulers. The simulation environment provided by this module is an important feature of the project.

4. Performance Analysis Module

The fourth and final module is the Performance Analysis Module. This module is responsible for evaluating and comparing the efficiency of different scheduling algorithms. It calculates important performance metrics such as average waiting time and average turnaround time for FCFS, Round Robin, and V-RTS-ML schedulers. The results are displayed in the form of tables and bar graphs for easy analysis. This module helps in demonstrating the superiority of the proposed machine learning-based scheduler over traditional methods. The graphical comparison generated by this module provides clear evidence of improved scheduling performance. All four modules are integrated into a single application to create a complete AI-assisted CPU scheduling simulator. The modular structure of the project ensures that each component functions independently while contributing to the overall objective of intelligent and adaptive CPU scheduling.

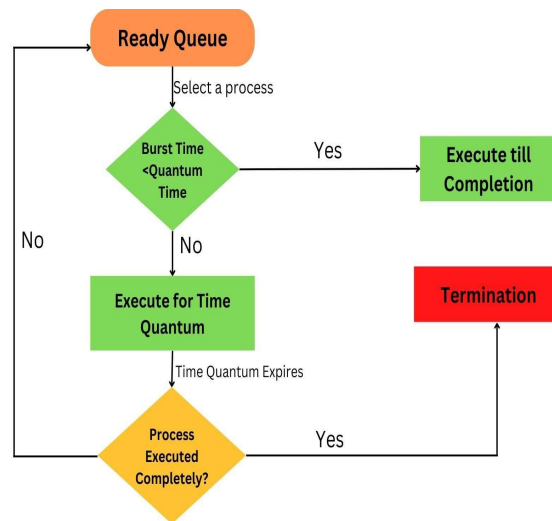


Figure 3.3 : FCFS

Chapter 4: Results and Recommendations

4.1 Overview of Implementation Results

After the successful implementation of the AI-assisted CPU Scheduling Simulator, various experiments were conducted to evaluate the performance of different scheduling algorithms. The system was tested using multiple sets of input processes containing different arrival times and burst times. The implemented algorithms namely First-Come-First-Serve (FCFS), Round Robin (RR), and the proposed V-RTS-ML scheduler were executed under identical conditions to ensure a fair comparison.

The simulator generated detailed scheduling outputs in the form of Gantt charts, performance tables, and graphical comparisons. These outputs were used to analyze how each algorithm behaves for different workloads. The results obtained from the system clearly demonstrate the advantages and limitations of traditional scheduling methods and highlight the improvements achieved through the machine learning-based scheduling approach. The performance of each algorithm was carefully observed and recorded using standard scheduling metrics such as waiting time and turnaround time. This chapter explains the experimental results obtained from the system and provides a detailed discussion on their significance and practical implications.

4.2 Results of FCFS Scheduling

The FCFS algorithm was executed using various input process combinations to analyze its behavior. As expected, FCFS strictly followed the order of process arrival without considering burst time or process priority. The Gantt charts generated for FCFS showed sequential execution of processes exactly in the arrival order. This behavior confirmed the theoretical working principle of the FCFS scheduling technique.

The performance metrics calculated for FCFS indicated higher average waiting time and turnaround time when long processes arrived earlier in the queue. This behavior confirmed the well-known limitation of FCFS known as the convoy effect, where short processes are forced to wait behind long processes. The results proved that FCFS is simple to implement but not suitable for dynamic workloads requiring better responsiveness. In several

test cases, it was observed that even a single long process at the beginning of the queue significantly affected the overall performance. These results clearly indicate that FCFS lacks flexibility and is inefficient for modern multitasking environments where process characteristics vary frequently.

4.3 Results of Round Robin Scheduling

The Round Robin scheduling algorithm was also tested using the simulator with different input scenarios. A fixed time quantum value was used for executing processes in cyclic order. The Gantt charts generated for Round Robin clearly illustrated the time-sliced execution of processes. Processes were executed for a limited duration and then moved back to the ready queue if not completed within the assigned quantum.

The experimental results showed that the performance of Round Robin depended heavily on the selected time quantum. When the quantum value was small, too many context switches occurred, leading to overhead and increased turnaround time. When the quantum value was large, the algorithm behaved similar to FCFS and lost its responsiveness. These results demonstrated the major drawback of Round Robin scheduling, which is its dependence on manually chosen static time quantum values. The experiments also showed that selecting an optimal quantum value manually is very difficult in real-world scenarios. Hence, Round Robin provides better fairness than FCFS but still lacks adaptability to different workloads and process behaviors.

4.4 Results of V-RTS-ML Scheduling

The proposed V-RTS-ML scheduler was executed using the same input processes to evaluate its effectiveness and efficiency. Unlike traditional algorithms, this scheduler used machine learning models to classify processes and predict optimal time quantum values dynamically. The Decision Tree Classifier identified whether a process was short or long, and the Decision Tree Regressor predicted an appropriate time quantum for long processes. These predictions were directly used by the scheduler to make intelligent execution decisions.

The results obtained from the V-RTS-ML scheduler showed significant improvement in scheduling efficiency. Short processes were identified and executed quickly, while long

processes were handled using adaptive time quantum values. The generated Gantt charts displayed more balanced and responsive execution compared to FCFS and Round Robin. The average waiting time and turnaround time were consistently lower for the V-RTS-ML scheduler in most test cases. These results confirmed that integrating machine learning into CPU scheduling improves decision making and overall system performance. The adaptive behavior of the scheduler allowed it to handle mixed workloads more effectively than static scheduling methods. Overall, the experimental observations proved that the proposed ML-based scheduler provides better performance, flexibility, and responsiveness than traditional CPU scheduling algorithms.

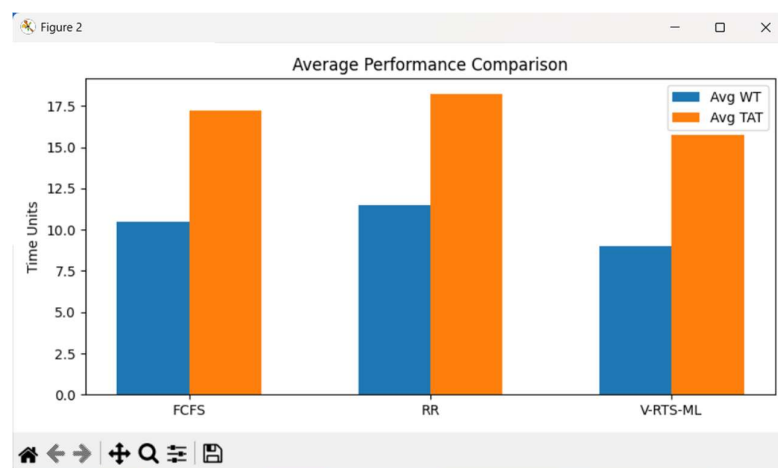


Figure 4.1: Performance Analysis

Chapter 5: Reflection on Learning and Personal Development

5.1 Key Learning Outcomes

The development of this project provided valuable learning experiences in both technical and analytical aspects of computer science and engineering. One of the major learning outcomes of this project was gaining a deep understanding of CPU scheduling concepts and how operating systems manage process execution. By implementing traditional scheduling algorithms such as FCFS and Round Robin, the fundamental working principles of scheduling mechanisms became much clearer. The practical implementation of these algorithms helped in understanding their advantages and limitations in real-world scenarios.

Another important learning outcome was the application of machine learning techniques to solve system-level problems. Integrating Decision Tree algorithms into CPU scheduling gave practical exposure to how artificial intelligence can be used to improve traditional computing methods. The project helped in understanding how data-driven decision making can enhance performance and adaptability in software systems. In addition, significant knowledge was gained in Python programming, graphical user interface development using Tkinter, and data visualization using Matplotlib. Overall, the project contributed to both theoretical understanding and practical technical skill development.

5.2 Challenges Encountered and Overcome

During the development of this project, several challenges were encountered at different stages. One of the initial challenges was understanding how to integrate machine learning models with CPU scheduling logic. Since traditional scheduling algorithms follow fixed rules, modifying them to work with dynamic ML predictions required careful planning and experimentation. Designing the V-RTS-ML scheduler in such a way that it could combine classification, regression, and scheduling logic was a complex task that required multiple iterations and testing.

Another challenge was the development of an interactive graphical user interface for

simulation. Creating a live scheduling simulation that visually represents process states and transitions was technically demanding. Handling real-time updates, Gantt chart generation, and performance comparison graphs required proper coordination between different modules. Debugging errors related to process execution flow and ensuring accurate calculation of waiting time and turnaround time also required considerable effort. Through continuous testing, learning from errors, and refining the logic, all these challenges were gradually overcome, resulting in a fully functional and stable system.

5.3 Application of Engineering Standards

Throughout the project, proper engineering standards and best practices were followed to ensure quality and reliability. A systematic software development approach was adopted, starting from requirement analysis, system design, coding, testing, and evaluation. The project was developed using modular programming techniques so that each component could be designed and tested independently. This approach improved the maintainability and readability of the code.

Standard programming conventions were followed while implementing algorithms and designing the graphical interface. Input validation and error handling mechanisms were included to make the system robust and user-friendly. Performance evaluation was carried out using well-defined metrics such as average waiting time and turnaround time, which are standard measures in operating system scheduling. Documentation of the project was also prepared in a structured and professional manner, following the academic report format provided by the institution. Adhering to these engineering standards helped in developing a high-quality and reliable simulation system.

5.4 Insights into the Industry

This project provided valuable insights into how real-world software systems are designed and optimized. In modern computing environments, intelligent decision-making plays a crucial role in improving system performance. The project demonstrated that even core operating system functions such as CPU scheduling can be enhanced using artificial intelligence and machine learning. This understanding is highly relevant to current industry trends, where automation and intelligent algorithms are increasingly being integrated into software applications.

Working on this project also highlighted the importance of developing user-friendly software tools that provide clear visualization and analysis capabilities. The simulator developed in this project can serve as a useful educational and analytical tool for students and professionals interested in operating systems and scheduling techniques. The experience gained from this project has strengthened problem-solving skills, programming abilities, and understanding of AI applications. These skills are highly valuable for future academic studies as well as professional careers in software development, system design, and artificial intelligence.

Chapter 6: Conclusion

The main objective of this project was to design and develop an intelligent CPU scheduling simulator that combines traditional scheduling algorithms with machine learning to improve efficiency, and this was successfully achieved through the AI-assisted CPU Scheduling Simulator. The project demonstrated that classical methods like First-Come-First-Serve and Round Robin can be enhanced using machine learning-based decision making, with the V-RTS-ML scheduler showing better performance than static approaches by dynamically classifying processes and predicting optimal time quantum values. It also identified key limitations of traditional methods, such as FCFS being unsuitable for dynamic workloads and Round Robin depending heavily on fixed time quantum, which justified the need for an adaptive solution. The simulator's graphical interface and real-time visualization using Gantt charts enabled users to compare algorithm performance, while experimental results confirmed reduced waiting and turnaround times with the proposed scheduler. Beyond technical outcomes, the project highlighted how AI can improve core operating system functions and demonstrated that intelligent scheduling can significantly enhance system performance, providing a practical tool for learning, research, and future development in intelligent resource management and system design.

Future Enhancements

- Extend the scheduler to support priority-based and real-time task constraints
- Train the decision tree model using larger and real-world workload datasets
- Include additional performance metrics such as CPU utilization and response time
- Implement multicore CPU scheduling support
- Deploy the simulator as a web-based or cloud application for wider accessibility
- Integrate deep learning models for more accurate scheduling decisions
- Enable adaptive learning where the model improves from runtime data
- Support dynamic workload prediction using AI techniques
- Implement energy-efficient scheduling for green computing environments
- Add support for distributed and parallel computing systems
- Integrate with real operating system kernels for practical deployment
- Develop a mobile or lightweight version for educational use
- Add advanced visualization dashboards for performance monitoring
- Introduce fault-tolerant and resilient scheduling mechanisms
- Automate parameter tuning for optimal scheduler performance

References

- Abbasi, M., & Shokrollahi, A. (2020). Enhancing the performance of decision tree-based packet classification algorithms using CPU cluster. *Cluster Computing*, 23(4), 3203-3219.
- Basaklar, T., Goksoy, A. A., Krishnakumar, A., Gumussoy, S., & Ogras, U. Y. (2023). DTRL: Decision tree-based multi-objective reinforcement learning for runtime task scheduling in domain-specific system-on-chips. *ACM Transactions on Embedded Computing Systems*, 22(5s), 1-22.
- Effah, E., Atsu, S. J., Brew, Z. A., Mensah, J. K., Quaicoe, E. A., Ansah, M. O., ... & Baffoe, R. P. (2025). Predicting CPU Burst Times with ML to Enhance Shortest Job First (SJF) and Shortest Remaining Time First (SRTF) CPU Scheduling. *International Journal of Computer Science and Information Security (IJCSIS)*, 23(5).
- Alsanea, M. (2021). Machine learning methods for CPU scheduling. *Trans. Mach. Learn. Data Min.*, 14(2), 47-59.
- Abraham Silberschatz, Peter Baer Galvin and Greg Gagne, “Operating System Concepts”, 10th Edition, John Wiley and Sons Inc., 2018.
- William Stallings, “Operating Systems – Internals and Design Principles”, 9th Edition, Prentice Hall, 2021.
- Andrew S. Tanenbaum, “Modern Operating Systems”, 5th Edition, Addison Wesley, 2022.

Appendices

Appendix A –

```
def v_rts_ml_scheduler(processes):
    time_now = 0
    gantt = []
    plist = sorted(processes, key=lambda x: x.at)
    completed = 0
    n = len(plist)
    long_rr_queue = deque()

    while completed < n:
        ready = [p for p in plist if p.at <= time_now and p.remaining > 0]

        if not ready:
            time_now += 1
            continue

        short = [p for p in ready if should_split_ml(p) == 0]
        long = [p for p in ready if should_split_ml(p) == 1]

        if short:
            p = min(short, key=lambda x: x.remaining)
            start = time_now
            time_now += 1
            p.remaining -= 1
        else:
            for lp in long:
                if lp not in long_rr_queue:
                    long_rr_queue.append(lp)

            if not long_rr_queue:
                time_now += 1
```

```

        continue

    p = long_rr_queue.popleft()
    q = get_ml_quantum(p)
    run = min(q, p.remaining)
    start = time_now
    time_now += run
    p.remaining -= run

    if p.remaining > 0:
        long_rr_queue.append(p)

    if gantt and gantt[-1][0] == p.pid:
        gantt[-1] = (p.pid, gantt[-1][1], time_now)
    else:
        gantt.append((p.pid, start, time_now))

    if p.remaining == 0:
        completed += 1
        p.ct = time_now
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt
        if p in long_rr_queue:
            long_rr_queue.remove(p)

    return gantt

```

Appendix B –

